

PHYS-467 Machine Learning for Physicists
Graded Homework n. 3
Regression with Random Features and Neural Networks
Due: December 23, 2021, 23:59

Instructions

- Please, upload your solution to the Moodle page of the course. No delay is accepted.
- The solution must be **submitted** on moodle: drafts will not be graded.
- The solution must consist of one notebook (.ipynb file) containing both the code and the written comments.
- The file should be called surname_name.ipynb. (e.g. Giovanni Piccioli $\mapsto$ piccioli_giovanni.ipynb). Use only characters in [a-z] (e.g. à, ç, é $\mapsto$ a, c, e) and use an underscore instead of the space.

In this homework, we will compare the performance of random-feature models and shallow fully-connected neural networks on a regression problem.

1. Download the dataset `hw3_dataset.pt` from moodle and load it in your notebook using the function `torch.load('hw3_dataset.pt')`, that returns a dictionary with keys `train_points`, `train_labels`, `test_points`, `test_labels`. Copy the four dictionary values in four different PyTorch tensors. What is the dimension of the training dataset? What is the dimension of the input space?
2. First, we try to solve the problem by using a random-feature network, which corresponds to a two-layer fully-connected network with the weights of the first layer fixed at their initial random values. Map the inputs X_μ into the features

$$\phi(X_\mu) = \sigma(W^\top X_\mu), \quad (1)$$

where $\sigma(u) = \max(0, u)$ is the ReLU activation function (that acts component-wise) and $W \in \mathbb{R}^{d \times p_{\text{RF}}}$ is a weight matrix with Gaussian random i.i.d. entries sampled from $\mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{d})$. Use $p_{\text{RF}} \in \{10, 100, 1000\}$ random features.

3. Using PyTorch, define a `nn.Sequential` model that takes a p_{RF} -dimensional input (the features $\phi(X_\mu)$) and applies a bias-less linear transformation, returning a scalar output.
4. Initialize the models and train them on the features $\phi(X_\mu)$ for 10^4 epochs minimizing the square loss using full-batch gradient descent. Try different learning rates $\eta = 10^k$, with $k \in \{0, -1, -2, -3, -4, -5\}$ and select the appropriate one for each value of p_{RF} . In order to select it, look for the one that does not blow up the training loss and at the same time gives the fastest decrease of the training loss.
5. For the selected dynamics, report the final test errors and plot the train and test losses as a function of the number of epochs. Comment.
6. Now, we're going to change the model, in particular we will train both layers. Using PyTorch, define a `nn.Sequential` model for a two-layer fully-connected network with ReLU activations, two units in the hidden layer and without biases

$$f(X_\mu) = W_2^\top \sigma(W_1^\top X_\mu), \quad (2)$$

where σ is the ReLU activation, $W_1 \in \mathbb{R}^{d \times p}$, $W_2 \in \mathbb{R}^{p \times 1}$, and $p = 2$.

7. Train this model on the inputs X_μ for 10^4 epochs minimizing the square loss using full-batch gradient descent. As before, try different learning rates in the same range, and select the appropriate one.¹
8. For the selected dynamics, report the final test error and plot the train and test losses as a function of the number of epochs. Comment.
9. Compare the test errors obtained in the two cases, random features and shallow neural network. Which of the two performs best? Comment.
10. Repeat the points 7. and 8. using $p = 100$, and increasing the number of epochs to 10^5 . Compare the error with the smaller neural network. Does this model achieve better or worse generalization?

¹As a good practice, especially for small models like this one, try different initializations in order to ensure your results are not due to a very-(un)lucky set of initial random weights.